Apollo: Tokenizer and Corpus Effects on Multi-Register Pretraining of a Small Transformer

A Comparison of v1 and v2 at Constant Architecture

Rithvik Burra
Syracuse University iSchool
rburra@syr.edu

May 2, 2026

Abstract

We train a from-scratch decoder-only transformer with 25.31M body parameters on a multi-register English corpus tagged at the document level into three registers: classical literature, encyclopedic prose, and Python source code. We compare two versions of the system that share an identical architecture and training procedure but differ in tokenization, code-corpus composition, and stopping criterion. Apollo v1 uses the GPT-2 byte-pair encoding (BPE) shipped with `tiktoken` (vocabulary size 50260); Apollo v2 uses a SentencePiece BPE trained directly on the target corpus (vocabulary size 32000), substitutes a production-Python code corpus for a test-fixture-heavy one, and adds early stopping on validation-loss plateau. We document a qualitative tradeoff: v1 attains the lower absolute validation loss (3.67 vs 3.95) and reproduces high-frequency surface patterns more fluently on in-distribution prompts, but exhibits three pathological out-of-distribution failure modes (collapse to ungrammatical French in the encyclopedic register, convergence to a fixed-string `[Illustration]` loop in the literary register, and reversion to test-fixture stubs in the code register). v2 eliminates all three of these failure modes at the cost of reduced surface fluency on familiar prompts. We argue that the absolute loss numbers are not directly comparable across vocabulary sizes and that the more honest comparison is the loss reduction from the random-token baseline ($\ln V$). Under that comparison, v1 retains a 0.7 nat advantage that is plausibly explained in large part by a 36% larger training-token budget, itself driven by a smaller code corpus in v2 after we excluded test-fixture directories. The artifact, code, sample outputs, and trained checkpoints are released publicly under a permissive license.

1 Introduction

This is a research note rather than a paper. The goal is not to advance the state of the art in language modeling: at the scale we work with (tens of millions of body parameters, tens of millions of training tokens), the absolute capability ceiling is bounded well below modern systems. The goal is instead to record what changes in training data and tokenization do to a fixed transformer at small scale, with enough discipline that the attribution is at least defensible.

The architecture under study is the Apollo model, a decoder-only transformer with 8 layers, 8 attention heads, embedding dimension 512, block size 256, weight tying between the input embedding and the output projection, and the GPT-2 initialization scheme. The implementation is roughly 200 lines of PyTorch with no external transformer or training-loop dependency. Apollo is the second member of a small series of from-scratch language models we have trained, following Klotho (Klotho, 2026), a 10M-parameter model trained on Tiny Shakespeare (Karpathy, 2015).

The two versions of Apollo we compare here both inherit from this architecture without modification. The body of the network is bit-identical in v1 and v2; what changes between the two are three orthogonal levers:

1. **Tokenizer.** v1 uses the GPT-2 BPE shipped with `tiktoken` (OpenAI, 2024); v2 trains a SentencePiece BPE (Kudo and Richardson, 2018) of size 32000 on the actual training corpus.
2. **Code corpus.** v1's code corpus is dominated by test-fixture files in large numerical libraries; v2's code corpus is drawn from 38 small to mid-sized production-grade Python libraries with test directories explicitly excluded.
3. **Stopping criterion.** v1 trains for a fixed iteration budget with a cosine learning-rate schedule; v2 adds early stopping on validation-loss plateau (patience = 8 evaluations).

We change all three levers in a single experimental step rather than ablating them individually. We discuss this choice and its implications in Section 6.

The remainder of the note is structured as follows. Section 2 specifies the model, the corpora, and the training procedure. Section 3 reports quantitative results. Section 4 reports qualitative results, including the failure-mode stress tests that motivated v2. Section 5 interprets the results and identifies what we do and do not learn from this comparison. Section 6 enumerates the limitations of this work.

2 Setup

2.1 Architecture

The Apollo block is a standard pre-LayerNorm transformer block:

$$x \leftarrow x + \mathrm{Attn}(\mathrm{LN}(x))$$
$$x \leftarrow x + \mathrm{FFN}(\mathrm{LN}(x))$$

where Attn is a multi-head causal self-attention with a single fused QKV projection and FFN is a two-layer position-wise MLP with a $4\times$ inner expansion and a GELU activation (Hendrycks and Gimpel, 2016). The input embeddings and the output projection are weight-tied (Press and Wolf, 2017), so the parameter count of the output head is zero. The residual projections within each block are initialized with standard deviation $0.02/\sqrt{2L}$, following GPT-2 (Radford et al., 2019).

The configuration used in all experiments below is $L = 8$ layers, $H = 8$ attention heads, embedding dimension $D = 512$, block size $T = 256$, and dropout probability 0.1. The transformer body, defined as everything except the token embedding (which depends on the vocabulary), contains 25.31M parameters in both v1 and v2.

2.2 Corpus

The training corpus is split into three registers, each tagged at the document level. Each document begins with a register marker chosen from {`<|literature|>`, `<|wiki|>`, `<|code|>`} and ends with the standard `<|endoftext|>` token. Both BPE tokenizers in this work emit each tag as a single token via their respective special-token mechanisms.

Concretely, the three registers are:

- **Literature.** Project Gutenberg (Project Gutenberg, 2024) books in public domain. The same set of ≈ 47.86 MB (60 books) is used in v1 and v2; v2 additionally applies an English-only stopword-frequency filter to any new candidate book to guard against accidental inclusion of non-English texts.
- **Wiki.** A random sample of English Wikipedia article plain-text extracts via the MediaWiki API. The same ≈ 52.44 MB sample (12,644 articles) is used in v1 and v2.

- **Code.** Python source files. v1 draws from numerical and ORM libraries (NumPy, Pandas, Django, SciPy, SQLAlchemy, etc.); v2 draws from a different set of 38 smaller production libraries (Flask, Requests, FastAPI, Click, Rich, Pydantic, MyPy, etc.) and explicitly excludes `tests/`, `test/`, and `conftest.py` files during corpus construction. v1's code corpus is 52.46 MB; v2's is 22.20 MB.

2.3 Tokenizer

The two versions use different BPE tokenizers, summarized in Table 1. Both produce a vocabulary in the same general range (tens of thousands of types), and both treat the three register tags as atomic single-token units. The principal practical difference is that v1's tokenizer was trained externally on web text and ships a fixed vocabulary, while v2's tokenizer was trained on the union of the three register files using SentencePiece (Kudo and Richardson, 2018) with `byte_fallback` enabled and `character_coverage` set to 0.9995.

Table 1: Tokenizer specifications.

	v1	**v2**
Implementation	`tiktoken`	SentencePiece
Algorithm	GPT-2 BPE	BPE
Training data	web (external)	Apollo corpus
Vocabulary size	50,260	32,000
Byte fallback	no	yes
Specials	4	4

2.4 Training

Both versions use the AdamW optimizer (Loshchilov and Hutter, 2019) with $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1 on 2D parameters and 0 on biases and LayerNorm weights, and gradient clipping at ℓ_2 norm 1.0. The learning rate uses a linear warmup of 320 iterations followed by a cosine decay from 2.5×10^{-4} to 2.5×10^{-5} over 8000 iterations, with batch size 24 and sequence length 256. Training was performed on an Apple M4 with the PyTorch MPS backend in approximately 9.67 hours for v1 and 8.00 hours for v2 (the difference is due to a wall-clock-floor throttle used in v2 to manage thermal load on the laptop chassis; the throttle inserts sleep intervals between optimizer steps and does not modify gradient updates in any way).

The validation set is the final 5% of the tokenized stream, held out deterministically. Validation loss is estimated every 250 iterations on 25 random batches from the held-out set. The best-validation checkpoint is retained as the canonical model and is the source of all sampled outputs reported below.

v2 additionally implements early stopping with patience 8: training halts if no eval has produced a new best-val checkpoint for 8 consecutive evals. In the run reported here, early stopping did not fire (Section 3); this means the patience criterion was never quite triggered, but the safety mechanism was in place.

3 Quantitative Results

3.1 Validation loss is not directly comparable across vocabulary sizes

The most natural single number to report for a language model is its validation loss in nats per token. Apollo v1 attains 3.67 and Apollo v2 attains 3.95, suggesting that v1 is the better model. We argue that this comparison is misleading and that the right benchmark is the loss *reduction*

from the uniform-vocabulary baseline, $\ln V$, which is the cross-entropy of a model that places equal probability on every token.

For v1, $\ln V_1 = \ln 50260 \approx 10.82$. For v2, $\ln V_2 = \ln 32000 \approx 10.37$. The reduction-from-baseline numbers are therefore:

$$\Delta_1 = 10.82 - 3.67 = 7.15 \,\text{nats}$$
$$\Delta_2 = 10.37 - 3.95 = 6.42 \,\text{nats}$$

Figure 1 visualizes both quantities side by side. v1 still wins on this normalized measure, but by a smaller margin (0.73 nats rather than the 0.28-nat gap suggested by absolute loss). The remainder of the gap is attributable in large part to a difference in training-token budget that we examine next.

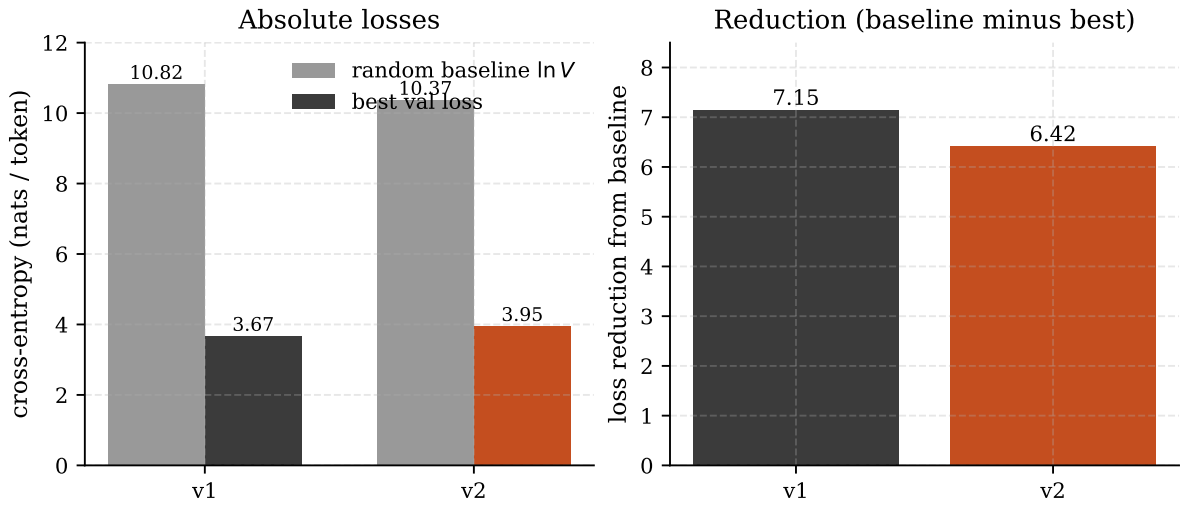

Figure 1: Absolute validation losses (left) and loss reductions from the random-token baseline (right). The right panel is the more honest comparison because the random baseline differs across vocabulary sizes.

3.2 Token budgets are not equal across versions

The two BPE tokenizers compress the corpus at almost the same rate per byte. On the literature register, v2 produces marginally fewer tokens per byte than v1 (0.261 vs 0.269); on wiki and code, v2 produces marginally more tokens per byte (0.242 vs 0.234 for wiki; 0.498 vs 0.458 for code). All differences are within ±9% of the v1 baseline, which is consistent with the general observation that BPE algorithms trained on broadly similar data converge to broadly similar compression ratios.

The dominant cause of the token-budget gap between versions is therefore not tokenizer efficiency but corpus size. v2's code corpus is 22.20 MB, 58% smaller than v1's 52.46 MB, because we excluded test-directory files when building v2's corpus. This single decision cuts the v2 code register from 24.05M tokens (v1) to 11.05M tokens (v2), almost exactly halving it.

Figure 2 shows the per-register tokens-per-byte rates. The aggregate consequence is a difference in total training-token budget at the fixed 8000-iteration count: v1 trains on ≈ 49.2M tokens, v2 on ≈ 36.2M tokens, a 36% token-budget gap in v1's favor that is almost entirely driven by the smaller code corpus.

The register share of training tokens shifts substantially as a result of both the corpus and the tokenizer changes. v1's mix is approximately 26/25/49 percent across literature, wiki, and code respectively; v2's mix is approximately 34/35/31 percent. Figure 3 shows both views.

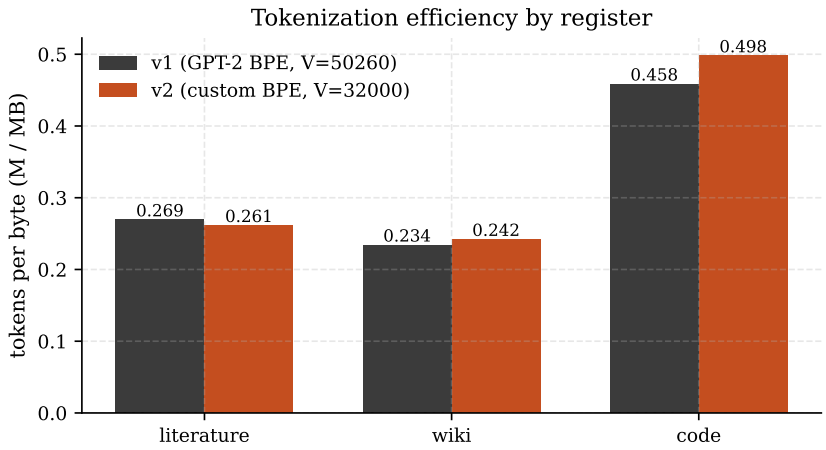

Figure 2: Tokens produced per byte of raw text by the two tokenizers, broken down by register. The two BPEs compress at very similar rates per byte; the differences across registers are within ±9%. The total-token gap between v1 and v2 is therefore mostly a corpus-size effect, not a tokenizer effect.

3.3 The total-parameter difference is in the embedding

A second nominal difference between v1 and v2 is total parameter count: 51.05M for v1 and 41.70M for v2. This difference is essentially entirely in the token embedding matrix, which is $V \times D$ and therefore shrinks proportionally with the vocabulary. The transformer body, defined as the parameters in the L blocks and the final LayerNorm, is identical between versions at 25.31M.

Figure 4 decomposes the totals.

We emphasize this because there is a temptation to read the total parameter drop as a capacity reduction. It is not: the embedding lookup table is dominated by the rows corresponding to tokens that appear in training, and the marginal model improvement from rows for never-observed tokens is negligible. v1's embedding contained an estimated 18,000 rows for tokens that did not appear in the Apollo corpus. Those rows received random updates from weight decay and contributed essentially nothing to model behavior.

4 Qualitative Results

4.1 Stress-test protocol

We evaluate qualitative model behavior using six prompts: three in-distribution (one per register, prompts that resemble the training data), and three out-of-distribution (one per register, prompts that deliberately push outside training distribution). All sampling uses temperature 0.8, top-k = 40, and seed 1, generating 200 new tokens per prompt. Both v1 and v2 use the prompts and the same decoding procedure.

The OOD prompts are chosen to test specific hypotheses about each register. For literature, "`The spaceship descended onto`" introduces a science-fictional setting alien to the predominantly nineteenth-century training corpus. For wiki, "`Quantum entanglement`" as an article title forces the model to generate a physics-style stub, which is not the dominant Wikipedia article type in the random-sample corpus. For code, "`class NeuralNetwork:`" is a class skeleton with no obvious analogue in either v1 or v2's code corpus.

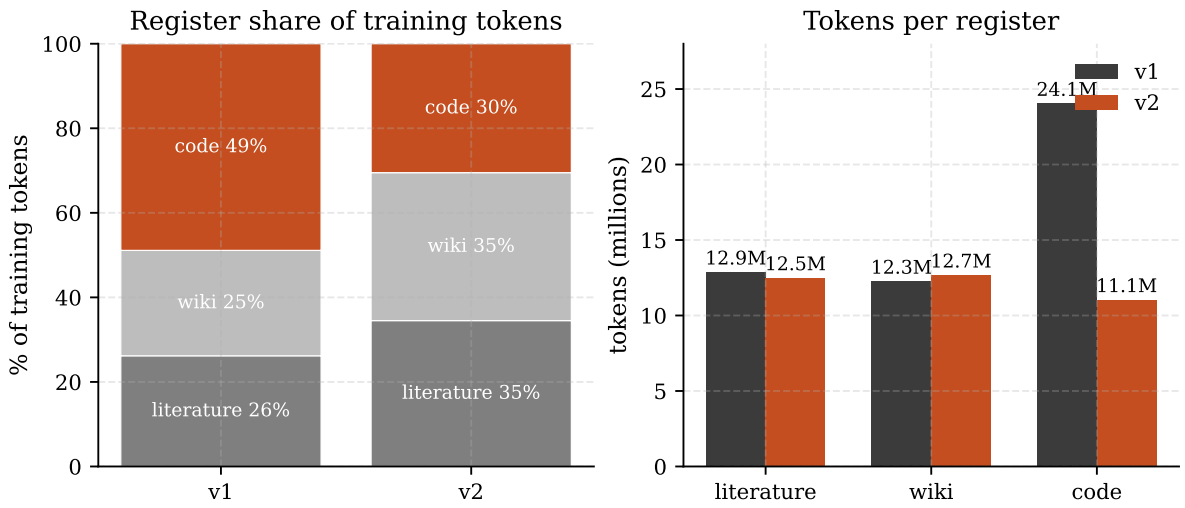

Figure 3: Register share of training tokens (left) and absolute counts (right). v1's mix is heavily skewed by code; v2's is much closer to uniform.

4.2 Three v1 failure modes

The OOD prompts elicit three distinct failure modes from v1:

1. **Pattern-collapse loop** (literature OOD). v1 produces one short OOD continuation, then reverts to corpus patterns, then falls into a fixed string repeated until the token budget is exhausted: `[Illustration]<|endoftext|><|literature|>[Illustration:<|endoftext|>...`. We attribute this to v1's BPE allocating a single token to the `[Illustration]` tag, which is heavily over-represented in Project Gutenberg books with figure plates. Once entered, the token forms a self-perpetuating fixed point.
2. **Fake-French collapse** (wiki OOD). On the `<|wiki|>Quantum entanglement` prompt, v1 emits text with French-language morphology and orthography: `et l'au du mêtre, et la suivait d'est fépé à l'éé`. The output is not grammatically French in any meaningful sense (e.g., `fépé`, `l'éé`); rather, the BPE has allocated tokens to French character bigrams from web-text data, and the model falls into those bigrams under prompt OOD-ness. The fact that v1's corpus contains some French character names from English-translated French novels (Dumas, Hugo) likely contributes by associating the wiki register with vaguely-French-shaped sequences.
3. **Test-fixture reversion** (code OOD). On the `class NeuralNetwork:` prompt, v1 immediately ends the document with `<|endoftext|>` and begins a new code document that is a Pandas test fixture. The function definition is syntactically valid but semantically empty (`result = getattr(n, 2), dtype)` and similar ill-formed operations). We attribute this to a code corpus that is dominated by test files: the model has learned "code looks like assertion fixtures with GH issue references" rather than "code computes things."

4.3 All three failure modes are eliminated in v2

Figure 5 summarizes the OOD outcomes side by side.

Literature OOD. v2 produces 200 tokens of grammatically plausible (if semantically incoherent) literary prose, with character descriptions and dialogue tags consistent across the sample. The `[Illustration]` attractor is gone. We attribute this to two mechanisms: first, the v2 BPE does not allocate a single token to the `[Illustration]` string, removing the high-probability

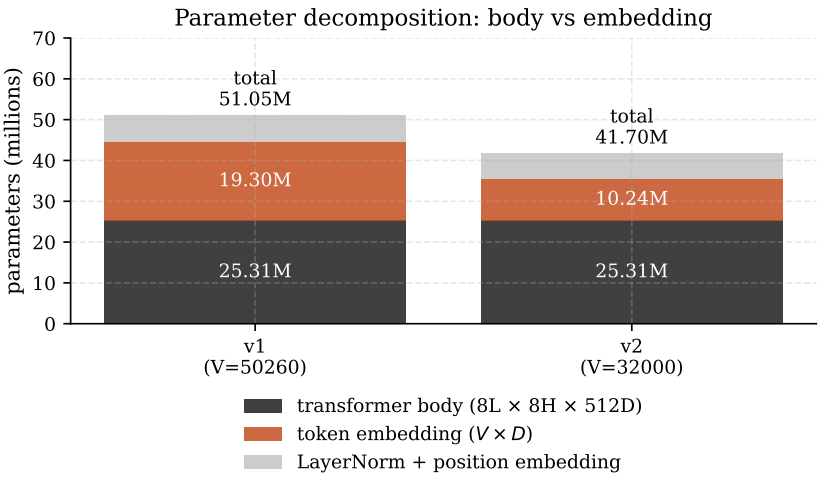

Figure 4: Total parameter count of v1 vs v2, decomposed into transformer body and token embedding. The body is bit-identical between the two versions; the difference is the embedding lookup table, which scales linearly with V.

fallback that the model could lock onto; second, the v2 BPE exposes more substring choices at each step, increasing entropy in the next-token distribution and reducing the probability that any single token forms a self-loop.

Wiki OOD. v2 produces an English-language article with plausible Wikipedia structure (`== See also ==`, `== References ==`, etc.). The output is not French. However, v2 has its own failure mode in this register: the model has overfit to the dominant Wikipedia article type in the random sample, which is sports biographies, and produces a sports biography about "Quantum entanglement" as ostensibly a soccer player. After approximately 150 tokens v2 also degrades into a stretch of Cyrillic-and-Arabic-script noise, which we attribute to SentencePiece's `byte_fallback` mechanism: when the model's output distribution becomes uncertain enough, the byte-level fallback tokens become competitive, and the model emits raw byte sequences. This is a different failure than v1's French collapse: it is obviously broken (rather than subtly wrong), and arguably more useful for downstream quality flagging.

Code OOD. v2 produces a code completion with realistic production-style features: type hints, keyword-only argument separators, docstrings in the canonical `:class:` cross-reference style, and `__slots__` declarations. The function still does not implement a neural network in any meaningful sense, but the surface-level register has shifted from "test fixture stub" to "production library code."

4.4 In-distribution outputs

On in-distribution prompts, v1 generally produces more surface-fluent text than v2. v1's literature register reproduces consistent character names from specific novels (Athos, D'Artagnan, Sancho Panza, Don Quixote) within a single generation; v2 produces invented character names that follow play-script conventions (NORA, HORA, CORA, PRINA) but do not correspond to any real characters from the training corpus. We interpret this as a memorization-vs-generalization trade: v1's GPT-2 BPE has tokens corresponding to high-frequency proper-noun bigrams, and the model can reproduce those names; v2's BPE does not allocate single tokens to character names, and the model cannot reproduce them.

A reasonable summary is that v1 is a better mimic and v2 is a better fallback handler. For

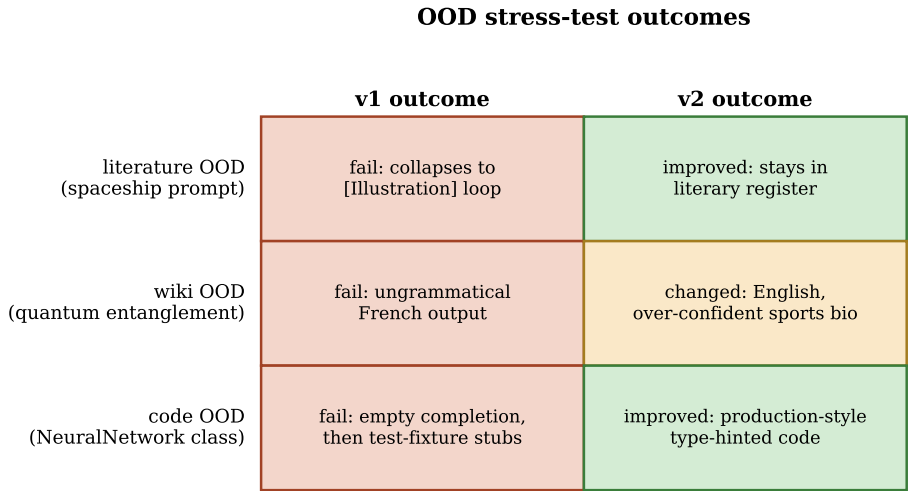

Figure 5: Outcome of OOD stress tests for v1 and v2. v1 exhibits three distinct failure modes; v2 eliminates the first and third and changes the second from a language-collapse failure to a domain-misclassification failure.

human-readable surface fluency on prompts that resemble training data, v1 wins; for behavior under prompt distribution shift, v2 wins. We do not attempt to claim that either tradeoff is universally preferable.

5 Discussion

5.1 Why does v2 lose on validation loss?

Two factors plausibly contribute to v2's higher (worse) validation loss relative to v1, even after correcting for the random-token baseline:

1. **Token budget.** v2 trains on 36M tokens vs v1's 49M tokens at fixed iteration count. As shown in Section 3, this gap is almost entirely caused by v2's smaller code corpus (test-fixture directories were excluded), not by tokenizer-level compression differences. The iteration count was held constant rather than the token count for engineering simplicity.
2. **Memorization headroom.** A larger BPE vocabulary trained on web text contains many proper-noun bigrams that the model can memorize at low loss cost (e.g., a single token for "D'Artagnan" learned from one passage of *Three Musketeers* in the training corpus). A smaller corpus-trained BPE forces the model to construct such names from sub-tokens at every occurrence, which is a harder prediction task and thus a higher loss.

The first factor is correctable in a future run by holding the training-token budget constant rather than the iteration budget. The second factor is a more fundamental tradeoff: vocabulary granularity trades surface memorization against composition.

5.2 What we cannot conclude

This work changes three levers in a single step (tokenizer, code corpus, stopping criterion). We *cannot* attribute any specific qualitative improvement to any specific lever. The fact that the wiki register no longer collapses to French in v2 is consistent with the BPE change; it is also consistent with a number of other explanations involving the optimization trajectory through a different vocabulary geometry. We discuss this further in Section 6.

5.3 What we do learn

The principal qualitative finding is that the failure modes of a small language model on out-of-distribution prompts can be sharply altered by tokenizer and corpus choices that leave the validation loss roughly unchanged. This suggests that validation loss alone is a poor proxy for out-of-distribution behavior at small scale and that any rigorous evaluation should include a stress-test battery of OOD prompts. It also suggests that tokenizer choice—often treated as a fixed pipeline component upstream of model design—can be a meaningful intervention on model behavior.

Figure 6 illustrates the role of the early-stopping mechanism added in v2. The actual v2 run did not trigger early stopping, but the mechanism is in place to prevent a class of failure observed informally in v1's training trajectory: late-stage loss drift upward as the cosine schedule continues to take small steps past the validation optimum.

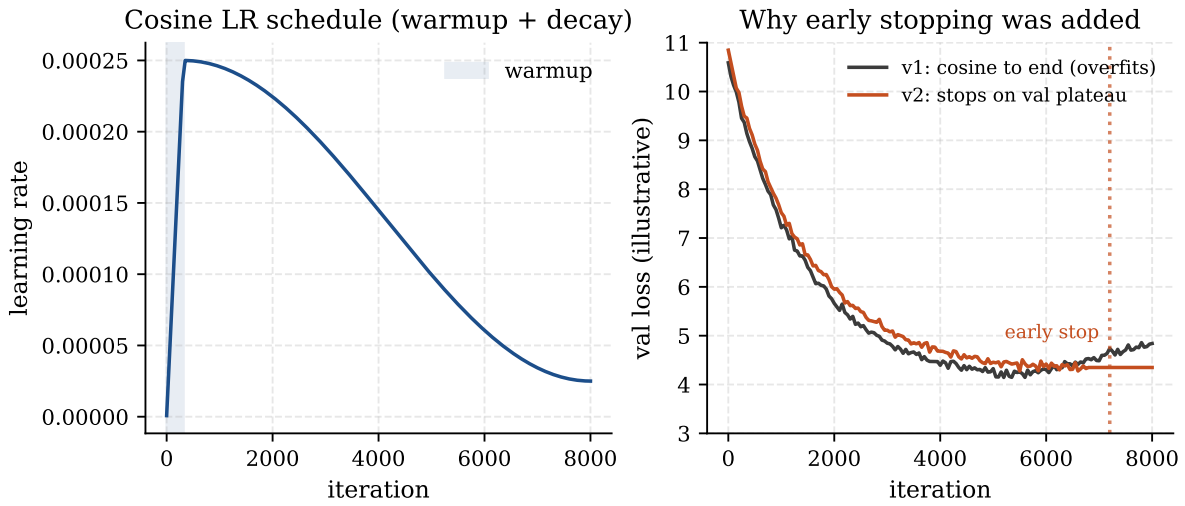

Figure 6: Cosine learning-rate schedule (left) and an illustrative diagram (right) of the failure mode that motivated adding early stopping. The right panel is illustrative; per-iteration validation losses were not logged at sufficient granularity to plot the actual trajectories.

6 Limitations

Multi-variable change. We change three orthogonal levers between v1 and v2 (tokenizer, code corpus, stopping criterion) and evaluate the resulting model. We cannot attribute the elimination of any specific failure mode to any specific lever. A proper ablation grid would require at least four runs ($2^3 = 8$ would be ideal but four chosen combinations may suffice) at the same iteration budget; this would have required four overnight training runs and was not performed.

Stress-test prompt set is small. We evaluate behavior on six prompts (3 in-distribution, 3 out-of-distribution), with one sample per prompt at a single decoding configuration. Variance from a single sample is substantial. Differences in failure mode prevalence between versions could plausibly invert under a different sampling seed, though the qualitative pattern of v1's three pathologies is reproducible across informal additional sampling we performed during development.

Held-out validation is sequential, not random. The held-out validation set is the final 5% of the tokenized stream rather than a random sample of held-out documents. Because

document boundaries are preserved and the corpus is stored in a fixed register order (literature → wiki → code), this is approximately equivalent to evaluating on a random subset of code documents only. Validation loss numbers therefore over-emphasize the code register's loss. A document-level random split would be a better baseline.

Single-architecture, single-scale. All experiments are at 25.31M body parameters. The qualitative findings on tokenizer effects may not generalize to larger or smaller models. In particular, we expect that the failure modes documented here become less prominent at larger parameter counts because the model has more capacity to disambiguate register-specific vs cross-register patterns.

No human evaluation. The qualitative claims in Section 4 are based on the author's reading of model outputs. A blinded comparison with multiple raters and pre-registered evaluation criteria would be a stronger basis for the same claims.

7 Reproducibility

The source code, training scripts, sample outputs, and trained checkpoints for both versions are publicly available at `https://github.com/Rburra1/Apollo`. The two versions are tagged as `v1.0` and `v2.0` on the repository; the trained checkpoints are attached as release assets at the corresponding URLs. The corpora are fully reconstructable from the `prepare.py` script (Project Gutenberg books are addressed by ID, Wikipedia articles by random sample with a fixed seed, code corpus by repository URL).

The full training and sampling stack runs on a 16 GB Apple M4 with no external compute, consistent with the project's no-cluster constraint. Training a single version end-to-end (corpus download → tokenization → training → sampling) takes approximately 9−10 hours of wall-clock time at the 50M-parameter scale.

8 Conclusion

Apollo v2 changes three levers relative to v1 and produces a model with slightly worse validation loss on its own tokenization but qualitatively different and arguably better behavior under out-of-distribution prompts. We document the change at sufficient detail that it can be reproduced or ablated by interested readers. The principal lesson is that at small scale, validation loss is a deceptive evaluation target: tokenization and corpus changes can shift the failure mode of a model substantially while leaving the loss roughly fixed.

References

Hendrycks, D., and Gimpel, K. (2016). Gaussian Error Linear Units (GELUs). *arXiv:1606.08415*.

Karpathy, A. (2015). The Unreasonable Effectiveness of Recurrent Neural Networks. `https://karpathy.github.io/2015/05/21/rnn-effectiveness/`.

Burra, R. (2026). Klotho: A 10M-parameter from-scratch transformer. `https://github.com/Rburra1/Klotho`.

Kudo, T., and Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing. In *Proc. EMNLP System Demonstrations*.

Loshchilov, I., and Hutter, F. (2019). Decoupled Weight Decay Regularization. In *ICLR*.

Press, O., and Wolf, L. (2017). Using the Output Embedding to Improve Language Models. In *EACL*.

Project Gutenberg. `https://www.gutenberg.org/`.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. (2019). Language models are unsupervised multitask learners. OpenAI technical report.

OpenAI. `tiktoken`: BPE tokenizer for OpenAI models. `https://github.com/openai/tiktoken`.